

I. VARIABLE AND FUNCTION DECLARATIONS

TABLE I
MAPPING BETWEEN NOTATION IN THE PAPER AND VARIABLES IN THE CODE.

Notation	Alias	Type	Comments
Self-declaration			
	solution	vector<double>	The type declaration for a solution and elastic point, i.e., s_i , x_i , and p_i .
	population	vector<solution>	The type declaration for a population and elastic points, i.e., s , x , and p .
Notation	Variable	Type	Comments
Struct: <region>			
$r_{i,k}$	rik	vector<int>	The index of the k -th elastic point associated with the i -th region.
$r_{i,b}$	rib	int	The index of the best elastic point in the region, used in the third term of Eq. 2.
I_i^a, I_i^b	ia, ib	int	The counters used in the first term of Eq. 2
	improvement	double	The improvement of an elastic point, used in the second term of Eq. 2.
	frib	double	The objective value of rib $f(r_{i,b})$, used in the third term of Eq. 2
e_i	ei	double	The expected value of a region as defined by Eq. 2.
Struct: <net>			
p	p	population	Assign a set of elastic points to the space net.
	fp	vector<double>	The objective value of an elastic point.
	improvement_p	vector<double>	The improvement of an elastic point, used in the second term of Eq. 2.
r	r	vector<region>	regions in the space net.
Variables			
runs	runs	int	The number of trials (runs) that will be carried out for an algorithm.
MaxFES	MaxFES	int	The maximum number of function evaluations.
d	d	int	The number of dimensions.
	net_size	int	It used to be the square root of the number of elastic points in the space net; that is, $\text{net_size} = \sqrt{n_p}$. Please see also the paper for more details.
n_s^{init}	ns_init	int	The initial number of explorers.
	ns_end	int	The minimum number of explorers.
n_x^{init}	nx_init	int	The initial number of miner.
	nx_end	int	The minimum number of miners.
	de_f	double	The scaling factor of the DE transition operator.
	de_cr	double	The crossover rate of the DE transition operator.
$n_{a,\text{max}}$	na_max	int	The number of elastic points that will be attracted by a new candidate solution.
	func_num	int	The index of the test function; e.g., 1 is for Ackley, 2 is for Griewank, and so on. Please see also test_function.h.
n_p	np	int	The number of elastic points.
h	nr	int	The number of regions.
$f(s_b)$	fbsf	double	The objective value of the best found solution.
s_b	bsf	solution	The best found solution.
	fbsf_avg	double	The average of the objective values of the best found solutions.
	fbsf_avg_FES	vector<double>	The average of the objective values of the best found solutions at all the FES.
	fbsf_run	vector<double>	The objective values of the best found solutions s_b in all the trials.
	fs	vector<double>	The objective values of explorers.
	fx	vector<double>	The objective values of miners.
L	min_bound	double	The lower bound of the search space.
U	max_bound	double	The upper bound of the search space.
	rg	mt19937_64	The random number generator, please see also https://en.wikipedia.org/wiki/Mersenne_Twister .

TABLE II
MAPPING BETWEEN SNO OPERATORS IN THE PAPER AND FUNCTIONS IN THE CODE.

Operator	Function	Return Type	Comments
Major Operators			
ExpectedValue()	ExpectedValue()	void	
RegionSearch()	RegionSearch()	void	
PointSearch()	PointSearch()	void	
SpaceNetAdjustment()	SpaceNetAdjustment()	bool	
PopulationAdjustment()	PopulationAdjustment()	void	
Initialization()	Initialization()	void	
Helping Functions			
	show_parameters()	void	
$f()$	evaluate()	double	
	roulette_wheel()	vector<pair<double, int>>	Build up a roulette wheel of probabilities for region selection. Used in RegionSearch().
	select_region()	int	Select a suitable region based on probabilities in the roulette wheel. Used in RegionSearch().
	get_ranks()	vector<int>	Get the ranks of the num_top_regions regions.
	dist()	double	Compute the Euclidean distance between two points; e.g., distance between an explorer and an elastic point.
	clamp_value()	double	Check to see if a searched solution is out of the boundary of the search space and clamp it if necessary.
	region_construct()	void	Divide the space net into regions. Used in Initialization().
	uniform_rand(void)	double	Generate a uniformly distributed random number in the range $[0, 1]$.
	uniform_rand(int last)	int	Generate a uniformly distributed random number in the range $[0 \dots last - 1]$.
	uniform_rand_pair(int last, int skip)	tuple<int, int>	Generate a pair of uniformly distributed random numbers that are different from not only each other but also skip if given and nonnegative.
	uniform_rand_in_bound()	double	Generate a uniformly distributed random number that is bounded by the boundary of the search space.
δ	delta()	double	Compute $\delta = FES/MaxFES$.